1. 项目一：MyTop —— 你的第一个 Linux 监控看板

这个项目的核心目的是教会你：**“如何用 Python 读取 Linux 底层数据，并像艺术品一样展示出来。”**

第一阶段：像外科医生一样准备环境

在 Linux 开发中，**环境隔离**是第一课。

1. 建立根据地

Bash

```
cd ~
mkdir MyTop
cd MyTop
```

- **详解**：我们在家目录建了一个专门的文件夹。这不仅是整洁，更是为了方便 Git 管理。

2. 初始化 Git (时光机)

Bash

```
git init
```

- **详解**：这个命令告诉 Git：“从现在开始，你要盯着这个文件夹里的每一个文件变化。”以后你改错代码了，随时能穿越回去。

3. 创建虚拟环境 (隔离手套) —— 关键步骤

Bash

```
python3 -m venv venv
```

- **是什么**：`venv` (virtual environment) 是 Python 的标准工具。它在你的项目里创建了一个 `venv` 文件夹，里面有一套独立的 Python 解释器。
- **为什么**：如果不做这步，你安装的库会跑到系统全局目录里。久而久之，你的系统里会有 100 个乱七八糟的库，甚至导致 Ubuntu 系统工具崩溃。

4. 激活环境 (穿上手套)

Bash

```
source venv/bin/activate
```

- **详解**：`source` 是读取脚本并执行。这句话的意思是：把刚才那个 `venv` 文件夹里的 Python 设置为当前终端的“默认 Python”。

- **标志**：你的命令行前面会出现 `(venv)`，说明你现在是在沙盒里操作，很安全。

5. 安装武器库

Bash

```
pip install psutil rich
```

- `psutil` **(Process and System Utilities)**：这是一个跨平台库，专门用来获取 CPU、内存、磁盘、网络温度等信息。它底层其实是在读取 Linux 的 `/proc` 文件夹。
- `rich`：这是一个极其强大的 UI 库。它能把终端丑陋的纯文本变成带颜色、带表格、带 emoji 的漂亮界面。

第二阶段：编写代码 (代码逐行精讲)

请打开 `main.py`。我把注释写得比代码还多，帮助你理解。

Python

```python
# --- 导入工具 ---
import time
import psutil
# 从 rich 库里导入我们需要的小组件：动态刷新、面板、表格、控制台
from rich.live import Live
from rich.panel import Panel
from rich.table import Table
from rich.console import Console
from datetime import datetime

# --- 核心逻辑区：获取数据 ---
def get_system_info():
    """
    这个函数负责当'搬运工'，从系统底层抓取数据，
    然后打包成一个字典返回。
    """

    # 1. 获取 CPU 使用率
    # interval=None 表示非阻塞模式，它会立刻返回上一次调用以来的平均值。
    # 如果填 1，程序会卡在这里等 1 秒来计算，会导致界面卡顿。
    cpu_usage = psutil.cpu_percent(interval=None)

    # 2. 获取内存信息
    memory = psutil.virtual_memory()
    # memory.total 单位是字节(Byte)，除以 1024^3 换算成 GB
    mem_total = memory.total / (1024 ** 3)
    mem_used = memory.used / (1024 ** 3)

    # 3. 获取开机时间
    # psutil.boot_time() 返回的是时间戳（一串数字），
    # datetime...strftime 把它变成人类能看懂的 "年-月-日 时:分:秒"
```

```python
    boot_time = datetime.fromtimestamp(psutil.boot_time()).strftime("%Y-%m-%d %H:%M:%S")

    # 把所有洗好的菜（数据）装盘（字典）返回
    return {
        "cpu": cpu_usage,
        "mem_total": f"{mem_total:.2f} GB", # .2f 表示保留两位小数
        "mem_used": f"{mem_used:.2f} GB",
        "mem_percent": memory.percent,
        "boot_time": boot_time
    }

# --- 视觉呈现区：画表格 ---
def generate_dashboard():
    """
    这个函数负责当'画家'，拿到数据，画出漂亮的表格。
    """
    data = get_system_info() # 先调用上面的函数拿到最新数据

    # 创建一个空表格，设定标题和颜色
    table = Table(title="MyTop System Monitor", style="cyan", header_style="bold magenta")

    # 定义三列：指标名、具体数值、视觉状态条
    table.add_column("指标", justify="right")
    table.add_column("数值", justify="left")
    table.add_column("状态", justify="center")

    # --- 填充第一行：CPU ---
    # 逻辑：如果 CPU 占用小于 50% 是绿色，否则是红色（警报）
    cpu_color = "[green]" if data['cpu'] < 50 else "[red]"
    # 技巧：'▇' * n 利用了字符串乘法，模拟进度条效果
    table.add_row(
        "CPU Usage",
        f"{data['cpu']}%",
        f"{cpu_color}{'▇' * int(data['cpu'] // 5)}"
    )

    # --- 填充第二行：内存 ---
    mem_color = "[green]" if data['mem_percent'] < 80 else "[red]"
    table.add_row(
        "Memory",
        f"{data['mem_used']} / {data['mem_total']}",
        f"{mem_color}{'▇' * int(data['mem_percent'] // 5)}"
    )

    # --- 填充第三行：开机时间 ---
    table.add_row("Boot Time", data['boot_time'], "✅")

    # 最后用 Panel (面板) 把表格包起来，加个蓝边框，显得专业
    return Panel(table, title="[bold yellow]Linux Dashboard", border_style="blue")

# --- 程序入口 ---
```

```
if __name__ == "__main__":
    console = Console()
    console.print("[bold green]正在启动监控...[/bold green]")
    time.sleep(1) # 假装在加载，增加仪式感

    try:
        # Live 是核心魔法：它会在屏幕同一位置不断重绘，而不是一直打印新行。
        # refresh_per_second=1：每秒刷新一次
        with Live(generate_dashboard(), refresh_per_second=1) as live:
            while True:
                # 死循环：只要不按 Ctrl+C，就一直跑
                # 每次循环都重新调用 generate_dashboard 获取新数据并重绘
                live.update(generate_dashboard())
                time.sleep(1) # 休息1秒，防止脚本自己把 CPU 跑满
    except KeyboardInterrupt:
        # 捕捉用户按下的 Ctrl+C 中断信号，优雅退出，不报错
        console.print("\n[bold red]监控已停止！[/bold red]")
```

项目二：Docker-Counter —— 真正理解微服务

这个项目不是为了写代码，而是为了理解**“容器编排”**。我们要把应用拆成两部分：**Web 服务器** 和 **数据库**，让它们分别在两个 Docker 容器里跑，并通过一根“隐形的网线”连接。

第一步：理解业务逻辑（`app.py`）

这就是一个简单的 Flask 网站，唯一的难点是连接 Redis。

Python

```python
import time
import redis
from flask import Flask

app = Flask(__name__)

# --- 关键点详解 ---
# cache = redis.Redis(host='redis', port=6379)
# 问：为什么 host 写 'redis'？我没在 hosts 文件里配置过这个域名啊？
# 答：这是 Docker Compose 的魔法。
#     在下面的 yaml 文件里，我们给数据库服务起名叫 'redis'。
#     Docker 会自动在内部 DNS 里把 'redis' 解析成那个容器的真实 IP。
cache = redis.Redis(host='redis', port=6379)

def get_hit_count():
    retries = 5
    while True:
        try:
            return cache.incr('hits') # 尝试让数据库里的 'hits' 数字加 1
        except redis.exceptions.ConnectionError as exc:
            # 问：为什么要重试？
            # 答：因为 Docker 启动时，Python 容器和 Redis 容器是同时启动的。
```

```
            #     有可能 Python 跑起来了，Redis 还没初始化好。
            #     所以如果连接失败，我们等 0.5 秒再试，一共试 5 次。
            if retries == 0:
                raise exc
            retries -= 1
            time.sleep(0.5)

@app.route('/')
def hello():
    count = get_hit_count()
    return f'Hello Ubuntu! 页面访问次数: {count}.\n'
```

第二步：理解 Dockerfile (构建说明书)

`Dockerfile` 是告诉 Docker 怎么把你的代码变成一个“镜像”。

Dockerfile

```
# 1. 地基：基于 Python 3.9 的精简版 Linux (Alpine 或 Slim)
#    这意味着容器里已经预装了 Python，不需要你操心。
FROM python:3.9-slim

# 2. 只有在此目录干活：以后所有命令都默认在 /app 下执行
WORKDIR /app

# 3. 复制依赖：先把 txt 拿进来，方便利用缓存
COPY requirements.txt requirements.txt

# 4. 安装：在容器内部执行 pip，把 Flask 和 Redis 库装进容器里
RUN pip install -r requirements.txt

# 5. 搬家：把当前电脑目录下的所有代码，复制到容器的 /app 目录
COPY . .

# 6. 声明：这个容器打算用 5000 端口（这只是个标记，不代表对外开放）
EXPOSE 5000

# 7. 启动令：当容器跑起来时，执行这句话
CMD ["python", "app.py"]
```

第三步：理解 Docker Compose (指挥官)

`docker-compose.yml` 是用来一次性管理多个容器的。

YAML

```
version: "3"
services:
  # --- 服务 A: 我们的 Python 网站 ---
  web:
    build: .  # 使用当前目录的 Dockerfile 现场构建镜像
    ports:
      - "8000:5000"
```

```yaml
      # 端口映射详解:
      # 左边 8000 是宿主机（你的 Ubuntu）端口
      # 右边 5000 是容器内部端口
      # 意思是: 当有人访问你 Ubuntu 的 8000 时, 把流量转发给容器的 5000。
    volumes:
      - .:/app
      # 挂载卷详解 (这一步对开发至关重要):
      # 意思是: 把你 Ubuntu 当前的代码目录, 直接“映射”到容器里的 /app。
      # 效果: 你在 VS Code 里修改了 app.py, 容器里会立刻生效, 不需要重新构建镜像!

  # --- 服务 B: Redis 数据库 ---
  redis:
    image: "redis:alpine"
    # 直接去 Docker Hub 下载官方做好的镜像, 不用我们自己写 Dockerfile
```

总结

- **MyTop 项目** 教会了你：Python 怎么调用系统命令、怎么做虚拟环境、怎么画终端界面。
- **Docker 项目** 教会了你：如何把复杂的环境依赖打包带走，以及服务之间是如何通过网络名称互相发现的。